PHASE 3 – DETAILED DESIGN

Project: Railway Reservation System – Concurrent Seat Allocation Simulator

---

STEP 1: PROJECT STRUCTURE DESIGN

1.1 Introduction

The project will follow a modular folder structure. Each module will have a specific responsibility, making the application easier to develop, test, debug and maintain.

The application will use Python Flask for backend development, SQLite for database storage, and HTML, CSS and JavaScript for the frontend.

1.2 Proposed Project Directory

```
None

Railway-Reservation-System/

|

├── app/

|   |

|   ├── __init__.py

|   ├── routes.py

|   ├── config.py

|   |

|   ├── models/
```

```
|   |   ├── __init__.py
|   |   ├── passenger.py
|   |   ├── journey.py
|   |   ├── seat.py
|   |   └── booking.py
|   |
|   ├── services/
|   |   ├── __init__.py
|   |   ├── booking_service.py
|   |   ├── seat_service.py
|   |   └── lock_manager.py
|   |
|   ├── database/
|   |   ├── __init__.py
|   |   ├── db_connection.py
|   |   ├── schema.sql
|   |   └── seed_data.py
|   |
|   ├── templates/
|   |   ├── index.html
|   |   ├── seats.html
```

```
|   |   ├── booking.html
|   |   ├── confirmation.html
|   |   └── admin.html
|   |
|   └── static/
|       ├── css/
|       |   └── style.css
|       |
|       ├── js/
|       |   └── script.js
|       |
|       └── images/
|
├── tests/
|   ├── test_seats.py
|   ├── test_booking.py
|   ├── test_concurrency.py
|   ├── test_api.py
|   └── test_performance.py
|
├── docs/
```

```
|   ├── requirements.md
|   ├── system_design.md
|   ├── detailed_design.md
|   └── test_cases.md
|
├── Dockerfile
├── docker-compose.yml
├── Jenkinsfile
├── requirements.txt
├── run.py
├── README.md
└── .gitignore
```

1.3 Folder Responsibilities

Folder/File	Responsibility
app/	Contains the main application
models/	Defines database entities
services/	Contains business logic
database/	Database connection and schema

templates/	HTML pages
static/	CSS, JavaScript and images
tests/	Automated test cases
docs/	Project documentation
Dockerfile	Defines application container
docker-compose.yml	Configures application services
Jenkinsfile	Defines CI/CD pipeline
requirements.txt	Lists Python dependencies
run.py	Starts the Flask application
README.md	Project setup and usage instructions

1.4 Design Decision

The application will separate database operations, business logic and user-interface logic.

For example, booking-related calculations and concurrency handling will be implemented in `booking_service.py`, rather than directly inside the Flask route.

Expected outcome: A modular project structure that supports implementation, testing and future maintenance.

STEP 2: DATABASE IMPLEMENTATION DESIGN

2.1 Introduction

The database will maintain passenger information, journey details, seat availability and booking records.

The main objective is to maintain data consistency and ensure that a seat cannot be confirmed for two passengers on the same journey.

2.2 Database Selection

Database: SQLite

SQLite is selected because:

- It is lightweight.
- It does not require a separate database server.
- It supports SQL transactions.
- It supports primary keys and foreign keys.
- It is suitable for the initial educational prototype.

2.3 Database Tables

The system will use five tables.

Table 1: Passengers

Column	Data Type	Constraint
passenger_id	INTEGER	Primary Key, Auto Increment

passenger_name	TEXT	NOT NULL
email	TEXT	UNIQUE, NOT NULL
phone	TEXT	Optional

Table 2: Journeys

Column	Data Type	Constraint
journey_id	INTEGER	Primary Key, Auto Increment
train_number	TEXT	NOT NULL
source	TEXT	NOT NULL
destination	TEXT	NOT NULL
journey_date	TEXT	NOT NULL

Table 3: Seats

Column	Data Type	Constraint
seat_id	INTEGER	Primary Key, Auto Increment

journey_id	INTEGER	Foreign Key
seat_number	TEXT	NOT NULL
seat_type	TEXT	NOT NULL
status	TEXT	NOT NULL

A seat number must be unique within a journey.

Table 4: Bookings

Column	Data Type	Constraint
booking_id	INTEGER	Primary Key, Auto Increment
passenger_id	INTEGER	Foreign Key
seat_id	INTEGER	Foreign Key
journey_id	INTEGER	Foreign Key
booking_status	TEXT	NOT NULL
booking_time	TEXT	NOT NULL

idempotency_key	TEXT	UNIQUE

The booking status can be:

- CONFIRMED
- REJECTED

Table 5: Booking Requests

Column	Data Type	Constraint
request_id	INTEGER	Primary Key, Auto Increment
passenger_id	INTEGER	Foreign Key
journey_id	INTEGER	Foreign Key
seat_id	INTEGER	Foreign Key
request_status	TEXT	NOT NULL
request_time	TEXT	NOT NULL
processing_time_ms	REAL	Optional

This table helps maintain request-level records for monitoring and analysis.

2.4 SQL Database Schema

The following SQL represents the proposed database structure.

```sql
PRAGMA foreign_keys = ON;


CREATE TABLE passengers (
    passenger_id INTEGER PRIMARY KEY AUTOINCREMENT,
    passenger_name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    phone TEXT
);


CREATE TABLE journeys (
    journey_id INTEGER PRIMARY KEY AUTOINCREMENT,
    train_number TEXT NOT NULL,
    source TEXT NOT NULL,
    destination TEXT NOT NULL,
    journey_date TEXT NOT NULL
);


CREATE TABLE seats (
    seat_id INTEGER PRIMARY KEY AUTOINCREMENT,
    journey_id INTEGER NOT NULL,
    seat_number TEXT NOT NULL,
```

```
    seat_type TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'AVAILABLE'
        CHECK(status IN ('AVAILABLE', 'BOOKED')),

    UNIQUE(journey_id, seat_number),
    UNIQUE(journey_id, seat_id),

    FOREIGN KEY(journey_id)
        REFERENCES journeys(journey_id)
);

CREATE TABLE bookings (
    booking_id INTEGER PRIMARY KEY AUTOINCREMENT,
    passenger_id INTEGER NOT NULL,
    seat_id INTEGER NOT NULL,
    journey_id INTEGER NOT NULL,
    booking_status TEXT NOT NULL
        CHECK(booking_status IN ('CONFIRMED', 'REJECTED')),
    booking_time TEXT NOT NULL,
    idempotency_key TEXT UNIQUE,
```

```
    FOREIGN KEY(passenger_id)
        REFERENCES passengers(passenger_id),

    FOREIGN KEY(journey_id, seat_id)
        REFERENCES seats(journey_id, seat_id)
);

CREATE UNIQUE INDEX unique_confirmed_seat_booking
ON bookings(journey_id, seat_id)
WHERE booking_status = 'CONFIRMED';

CREATE TABLE booking_requests (
    request_id INTEGER PRIMARY KEY AUTOINCREMENT,
    passenger_id INTEGER NOT NULL,
    journey_id INTEGER NOT NULL,
    seat_id INTEGER NOT NULL,
    request_status TEXT NOT NULL,
    request_time TEXT NOT NULL,
    processing_time_ms REAL,

    FOREIGN KEY(passenger_id)
```

```
        REFERENCES passengers(passenger_id),


    FOREIGN KEY(journey_id, seat_id)
        REFERENCES seats(journey_id, seat_id)
);
```

2.5 Database Integrity

The following database rules will be enforced:

1. Every passenger must have a unique passenger ID.
2. Every journey must have a unique journey ID.
3. Seat numbers must be unique within a journey.
4. A seat must belong to an existing journey.
5. A booking must reference an existing passenger and seat.
6. Only one confirmed booking can exist for a particular seat and journey.
7. A rejected booking must not change the confirmed seat allocation.
8. An idempotency key must not be reused for a different booking request.

The partial unique index on confirmed bookings is particularly important because it prevents duplicate confirmed records even if an application-level concurrency check fails.

2.6 Database Relationship Summary

- Passenger → Booking: One-to-Many.
- Journey → Seat: One-to-Many.
- Journey → Booking: One-to-Many.
- Seat → Booking: One-to-Many over time, but only one confirmed booking at a time.
- Passenger → Booking Request: One-to-Many.

Expected outcome: A relational database design with primary keys, foreign keys, transactions and duplicate-booking protection.

STEP 3: CLASS-LEVEL DESIGN

3.1 Introduction

The system will use object-oriented programming to represent passengers, journeys, seats and bookings.

Each class will have clearly defined attributes and methods.

3.2 Passenger Class

Purpose: Represents a passenger using the reservation system.

Attributes:

- passenger_id
- passenger_name
- email
- phone

Methods:

- register()
- view_available_seats()
- make_booking()
- view_booking_history()

3.3 Journey Class

Purpose: Represents a train journey.

Attributes:

- journey_id
- train_number
- source
- destination
- journey_date

Methods:

- get_journey_details()
- get_available_seats()

3.4 Seat Class

Purpose: Represents an individual seat.

Attributes:

- seat_id
- journey_id
- seat_number
- seat_type
- status

Methods:

- check_availability()
- update_status()
- get_seat_details()

3.5 Booking Class

Purpose: Represents a booking request and its result.

Attributes:

- booking_id
- passenger_id
- seat_id
- journey_id
- booking_status
- booking_time

Methods:

- create_booking()
- confirm_booking()
- reject_booking()
- get_booking_details()

3.6 BookingService Class

Purpose: Coordinates the complete booking operation.

Methods:

- validate_request()
- process_booking()
- check_seat_availability()
- allocate_seat()
- confirm_booking()
- reject_booking()

3.7 LockManager Class

Purpose: Controls concurrent access to the booking operation within one application process.

Methods:

- acquire_lock()
- release_lock()

3.8 DatabaseManager Class

Purpose: Handles communication with the SQLite database.

Methods:

- connect()
- execute_query()
- save_booking()
- update_seat()
- retrieve_booking()
- close_connection()

3.9 Class Interaction Diagram

```
None
+------------------+

|    Passenger     |

+------------------+

| passenger_id     |

| passenger_name   |

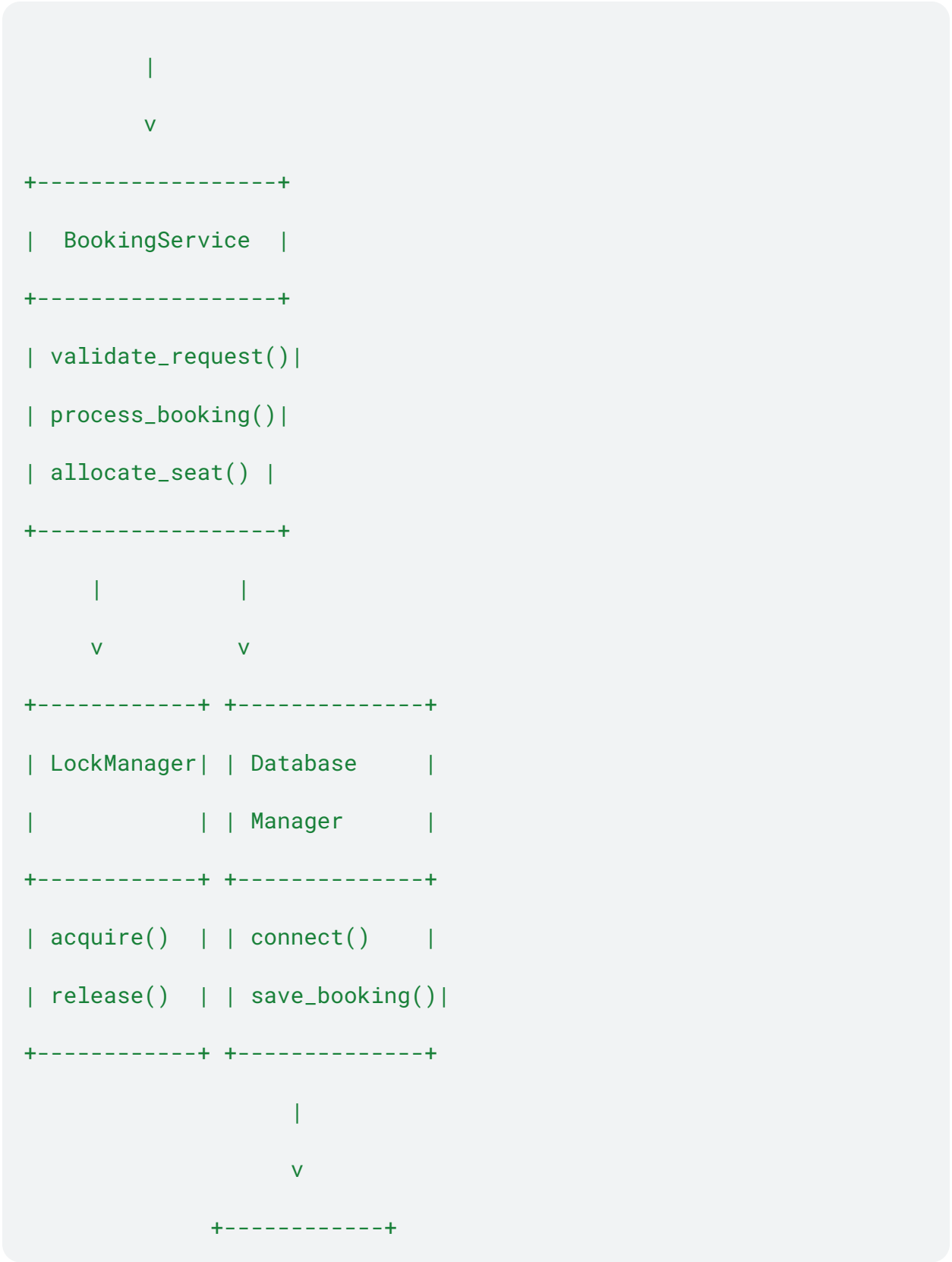
| email            |

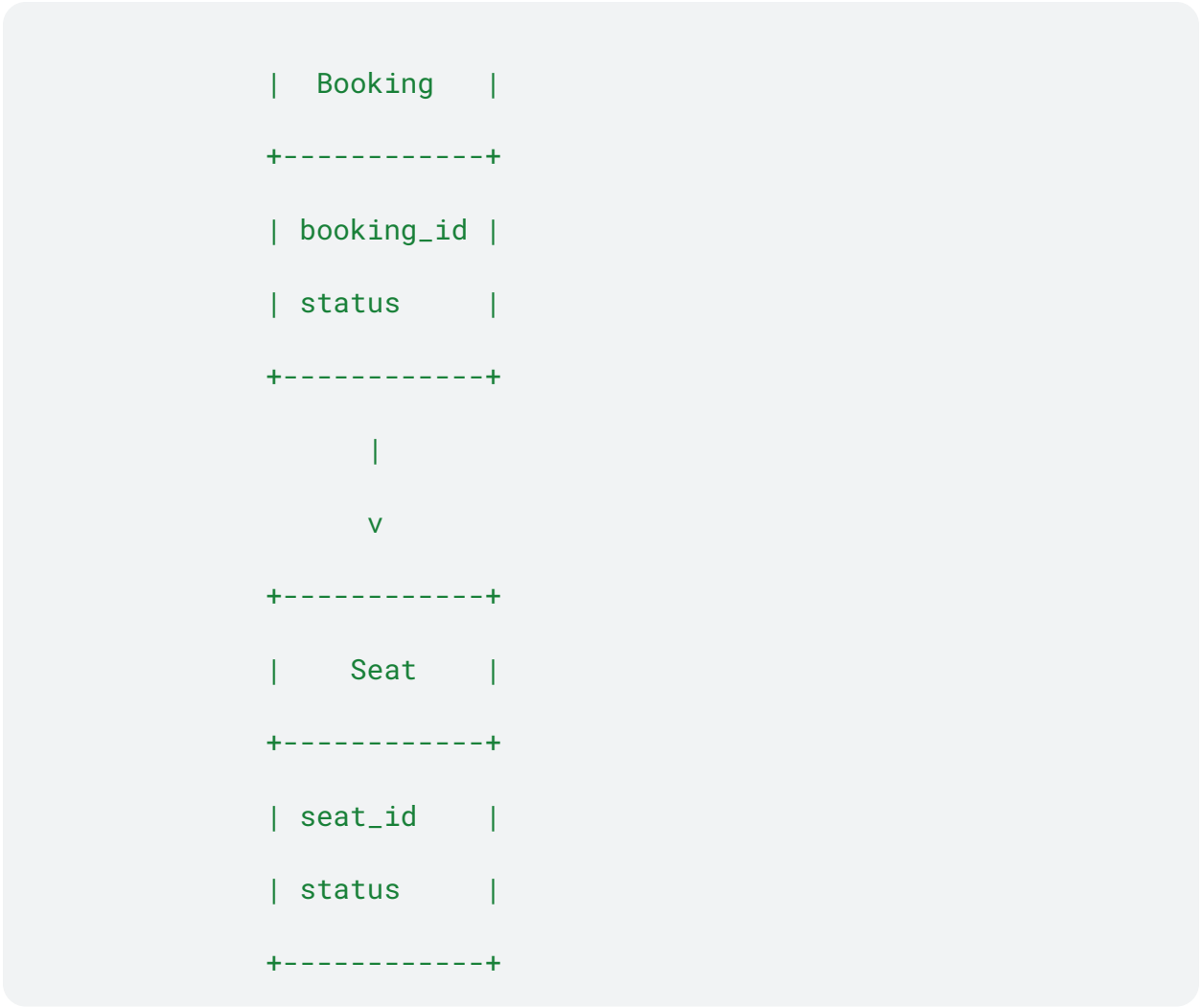
+------------------+

| make_booking()   |

| view_history()   |

+------------------+
```


|
v
+------------------+
| BookingService |
+------------------+
| validate_request()|
| process_booking()|
| allocate_seat() |
+------------------+
| |
v v
+------------+ +--------------+
| LockManager| | Database |
| | Manager |
+------------+ +--------------+
| acquire() | | connect() |
| release() | | save_booking()|
+------------+ +--------------+
|
v
+------------+



3.10 Design Decision

Business logic will be maintained in service classes, while database access will be handled separately.

This separation makes it easier to test the booking algorithm independently of the user interface.

Expected outcome: A clear object-oriented design with defined responsibilities for every class.

STEP 4: FUNCTION-LEVEL DESIGN

4.1 Introduction

This step defines the important functions required to implement the system.

Each function will have a defined purpose, input, processing logic and output.

4.2 Function Specifications

Function	Input	Responsibility	Output
register_passenger()	Passenger details	Register passenger	Passenger ID
get_journeys()	Search details	Retrieve journeys	Journey list
get_available_seats()	Journey ID	Retrieve available seats	Seat list
validate_request()	Passenger ID, Journey ID, Seat ID	Validate request	True/False
acquire_lock()	Lock object	Acquire lock	Lock status
check_seat_availability()	Journey ID, Seat ID	Check availability	True/False
allocate_seat()	Passenger ID, Journey ID, Seat ID	Allocate seat	Booking result
save_booking()	Booking details	Store booking	Booking ID
update_seat_status()	Seat ID, Status	Update seat	Success/Failure

get_booking_history()	Passenger ID	Retrieve booking history	Booking list
record_request()	Request details	Record request	Request ID

4.3 Function: get_available_seats()

Purpose: Retrieve all seats available for a particular journey.

Input:

- journey_id

Processing:

1. Validate journey ID.
2. Query the seats table.
3. Filter seats with status AVAILABLE.
4. Return the seat list.

Output:

- List of available seats.

4.4 Function: validate_request()

Purpose: Validate a booking request before processing it.

Input:

- passenger_id
- journey_id
- seat_id

Processing:

1. Check whether passenger exists.
2. Check whether journey exists.
3. Check whether seat belongs to that journey.
4. Check whether all required fields are valid.

Output:

- Valid request or validation error.

4.5 Function: allocate_seat()

Purpose: Safely allocate an available seat to a passenger.

Input:

- passenger_id
- journey_id
- seat_id
- idempotency_key

Processing:

1. Validate request.
2. Acquire concurrency lock.
3. Start database transaction.
4. Check for an existing request with the same idempotency key.
5. Check seat availability.
6. Insert booking record.
7. Update seat status.
8. Commit transaction.
9. Release lock.

Output:

- Confirmed booking or rejection response.

4.6 Function: update_seat_status()

Purpose: Update the seat's availability after booking.

Input:

- seat_id
- status

Processing:

1. Validate seat ID.
2. Update the seat status.
3. Confirm that the database operation succeeded.

Output:

- Updated status.

Expected outcome: Detailed specifications for the functions that will be implemented in Phase 4.

STEP 5: SEAT ALLOCATION ALGORITHM DESIGN

5.1 Introduction

Seat allocation is the central business operation of the Railway Reservation System.

The algorithm must verify seat availability and ensure that a seat is assigned to only one passenger for a particular journey.

5.2 Basic Booking Algorithm

```
None
START


Receive passenger ID, journey ID and seat ID.


Validate passenger information.


Validate journey information.


Validate seat information.


IF any validation fails:
```

```
    Return INVALID REQUEST.

Acquire concurrency lock.

Begin database transaction.

Check whether seat is AVAILABLE.

IF seat is AVAILABLE:

    Create booking record.

    Update seat status to BOOKED.

    Commit database transaction.

    Return BOOKING SUCCESSFUL.

ELSE:
```

```
    Roll back transaction.

    Return SEAT ALREADY BOOKED.

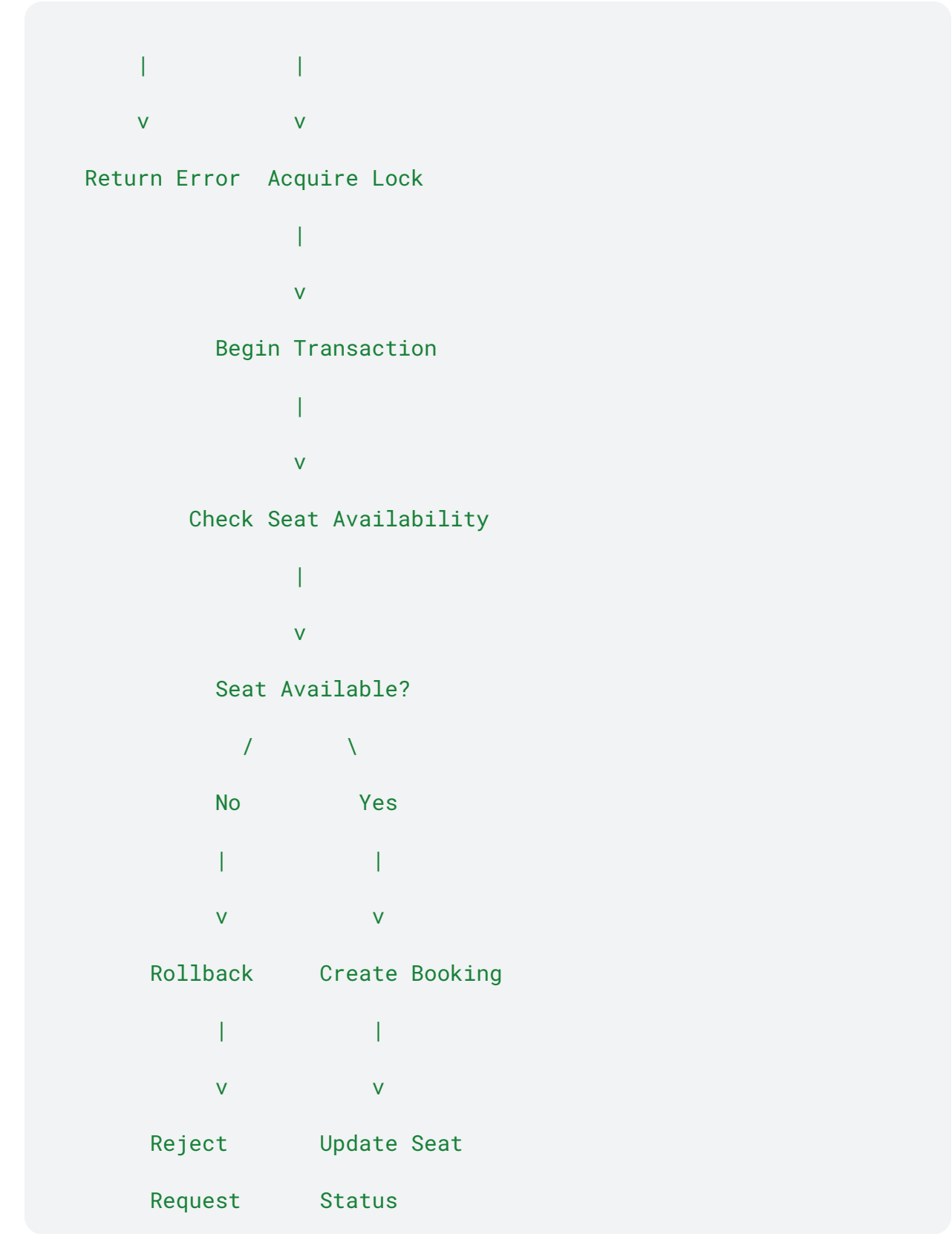
Release concurrency lock.

STOP
```

5.3 Booking Flowchart

None

```
          START
            |
            v
    Receive Booking Request
            |
            v
      Validate Request
            |
            v
      Is Request Valid?
         /       \
       No         Yes
```


|
|
v
v
Return Error
Acquire Lock
|
v
Begin Transaction
|
v
Check Seat Availability
|
v
Seat Available?
/
\
No
Yes
|
|
v
v
Rollback
Create Booking
|
|
v
v
Reject
Request
Update Seat
Status

```
                |
                v
            Commit
                |
                v
          Confirm Booking
                |
                v
         Release Lock
                |
                v
              END
```

5.4 Algorithm Complexity

Let:

- n = number of seats.
- m = number of booking records.

With appropriate database indexes:

- Seat lookup: approximately O(log n).
- Booking lookup by indexed key: approximately O(log m).
- Booking insertion: approximately O(log m), excluding database-specific overhead.

The complete request also involves transaction coordination, lock waiting and disk operations. Therefore, these complexity estimates do not establish the actual response time.

5.5 Design Decision

The booking operation will be treated as a critical transaction. The database must either commit the complete booking or roll back the changes.

A partially completed booking must never be returned as successful.

Expected outcome: A defined seat allocation algorithm that can be translated into Python code.

STEP 6: CONCURRENCY CONTROL ALGORITHM DESIGN

6.1 Introduction

The Tatkal booking scenario may involve thousands of users attempting to book seats simultaneously.

If multiple requests access the same seat without synchronization, a race condition may occur.

6.2 Race Condition Example

Assume only one seat remains available.

```
None
Passenger A                 Passenger B


Check seat                  Check seat


Seat available              Seat available


Book seat                   Book seat
```

```
Booking confirmed          Booking confirmed
```

Both passengers may receive confirmation if the application does not protect the booking operation.

This is unacceptable.

6.3 Proposed Concurrency Control

The prototype will use three protection mechanisms:

1. Python threading.Lock

Prevents multiple threads within the same application process from entering the protected booking section simultaneously.

2. Database transaction

Ensures that booking creation and seat-status updates happen atomically.

3. Database uniqueness constraint

Prevents more than one confirmed booking for the same seat and journey.

6.4 Detailed Concurrency Algorithm

```
None

START


Receive booking request.


Validate the request.


Acquire the application-level lock.
```

```
BEGIN IMMEDIATE DATABASE TRANSACTION.

Check whether the idempotency key was already processed.

IF previously processed:

    Return the existing result.

Check seat availability.

IF seat is AVAILABLE:

    Insert CONFIRMED booking.

    Update seat status to BOOKED.

    Commit transaction.

    Return SUCCESS.

ELSE:
```

```
    Roll back transaction.

    Return SEAT UNAVAILABLE.

IF database error occurs:

    Roll back transaction.

    Return controlled error.

Release lock.

END
```

6.5 Two-User Concurrency Example

Initial condition:

```
None
Seat Number: A1

Journey ID: 501

Status: AVAILABLE
```

Two requests arrive:

Time	Passenger A	Passenger B
T1	Sends booking request	Sends booking request
T2	Acquires lock	Waits for lock
T3	Begins transaction	Waiting
T4	Checks seat	Waiting
T5	Creates booking	Waiting
T6	Updates seat status	Waiting
T7	Commits transaction	Waiting
T8	Releases lock	Acquires lock
T9	Booking confirmed	Checks seat
T10	—	Seat already booked
T11	—	Booking rejected

Expected result:

```
None
Passenger A: CONFIRMED

Passenger B: REJECTED



Confirmed bookings: 1

Duplicate bookings: 0
```

6.6 Important Production Limitation

A Python threading lock works only within one application process.

If multiple Flask workers or servers are used, their in-memory locks are separate.

For multi-process or distributed production deployment, database-level transactions and constraints must protect seat allocation across all workers. A suitable distributed locking system may also be considered.

SQLite's write transactions serialize writers, which can limit throughput. The educational prototype will therefore not be assumed to support 50,000 attempts per minute.

Expected outcome: A concurrency-control design that prioritises correctness and prevents duplicate confirmed bookings.

STEP 7: API-LEVEL DESIGN

7.1 Introduction

The backend will provide REST APIs to allow the frontend to communicate with the Flask application.

The API layer will receive requests, validate inputs, call the relevant service and return structured responses.

7.2 API Endpoint Specifications

API ID	Method	Endpoint	Purpose
API-01	GET	/api/journeys	Retrieve journeys
API-02	GET	/api/seats/<journey_id>	Retrieve journey seats
API-03	GET	/api/seats/<journey_id>/available	Retrieve available seats
API-04	POST	/api/bookings	Create booking
API-05	GET	/api/bookings/<booking_id>	Retrieve booking details
API-06	GET	/api/passengers/<passenger_id>/bookings	Retrieve booking history
API-07	GET	/api/admin/bookings	Retrieve all bookings
API-08	GET	/api/admin/metrics	Retrieve monitoring metrics

7.3 Booking API Request

Endpoint:

```
None
POST /api/bookings
```

Request:

JSON

```
{
    "passenger_id": 101,
    "journey_id": 501,
    "seat_id": 25,
    "idempotency_key": "request-101-501-25-001"
}
```

7.4 Successful Response

JSON

```
{
    "status": "success",
    "message": "Seat booked successfully",
    "booking_id": 1001,
    "journey_id": 501,
    "seat_id": 25,
    "booking_status": "CONFIRMED"
}
```

7.5 Rejected Response

JSON

```
{
    "status": "failed",
    "message": "Seat is already booked",
```

```
    "booking_status": "REJECTED"
}
```

7.6 HTTP Status Codes

Code	Meaning
200	Successful request
201	Booking created
400	Invalid request
401	Authentication required
403	Access denied
404	Resource not found
409	Seat already booked or booking conflict
500	Internal server error
503	Temporary service unavailability

7.7 API Validation Rules

1. Passenger ID must exist.
2. Journey ID must exist.
3. Seat ID must belong to the specified journey.
4. Required request fields must not be empty.
5. The seat must be available.
6. Duplicate requests must be handled using idempotency.
7. Errors must return clear messages without exposing internal database details.

Expected outcome: A documented API structure for communication between the frontend and backend.

STEP 8: USER INTERFACE DESIGN

8.1 Introduction

The user interface will be designed to provide passengers with a simple booking experience.

The system will contain four primary screens.

8.2 Screen 1: Passenger Dashboard

Purpose:

- Display the application name.
- Allow journey search.
- Collect source and destination.
- Collect journey date.
- Display available journeys.

Proposed layout:

```
None
+-----------------------------------------+

|       RAILWAY RESERVATION SYSTEM         |

+-----------------------------------------+

|                                          |
```

```
|       Search Your Journey                |
|                                          |
| Source:       [____________]             |
| Destination:  [____________]             |
| Journey Date: [____________]             |
|                                          |
|            [ SEARCH TRAINS ]             |
|                                          |
+------------------------------------------+
```

8.3 Screen 2: Seat Selection

Purpose:

- Display the seats for the selected journey.
- Show seat availability.
- Allow the passenger to select a seat.
- Submit a booking request.

Proposed layout:

```
None
+------------------------------------------+
|              SELECT YOUR SEAT            |
+------------------------------------------+
|                                          |
|  AVAILABLE       BOOKED       SELECTED    |
|                                          |
```

```
|   [A1]            [A2]          [A3]      |
|   [B1]            [B2]          [B3]      |
|   [C1]            [C2]          [C3]      |
|                                           |
|                                           |
|              [ BOOK NOW ]                 |
+-------------------------------------------+
```

8.4 Screen 3: Booking Confirmation

Purpose:

- Display booking status.
- Display booking ID.
- Display selected journey and seat.
- Inform the passenger whether the booking succeeded or failed.

Proposed layout:

```
None
+-------------------------------------------+
|            BOOKING RESULT                 |
+-------------------------------------------+
|                                           |
|             BOOKING SUCCESSFUL            |
|                                           |
| Booking ID: 1001                          |
| Journey ID: 501                           |
```

```
| Seat Number: A1                          |
| Status: CONFIRMED                        |
|                                          |
|            [ BACK TO HOME ]              |
+------------------------------------------+
```

8.5 Screen 4: Admin Dashboard

Purpose:

- Display total booking requests.
- Display successful bookings.
- Display rejected requests.
- Display current seat availability.
- Display bookings per second.
- Display error rate.

Proposed layout:

```
None
+------------------------------------------+
|              ADMIN DASHBOARD             |
+------------------------------------------+
| Total Requests:       5000               |
| Confirmed Bookings:   4200               |
| Rejected Requests:     800               |
| Available Seats:      250                |
|                                          |
| Bookings Per Second:   85                |
```

```
| Error Rate:            2%                |
|                                          |
|             BOOKING RECORDS              |
|                                          |
| ID       Passenger       Seat    Status  |
| 1001     Passenger A     A1      CONFIRMED|
| 1002     Passenger B     A2      CONFIRMED|
+------------------------------------------+
```

The values shown above are illustrative, not measured system results.

8.6 UI Design Principles

- Simple navigation.
- Clear seat availability indicators.
- Responsive layout.
- Clear booking status messages.
- Input validation.
- No unnecessary personal information displayed.
- Accessible text and controls.

Expected outcome: Four initial interface specifications for implementation in Phase 4.

STEP 9: TEST CASE DESIGN

9.1 Introduction

Testing will be planned during the design stage according to the V-Model.

The system must be tested for functional correctness, database consistency, concurrency behaviour, security and performance.

9.2 Unit Test Cases

Test ID	Test Description	Input	Expected Result
TC-01	Validate passenger	Existing passenger ID	Valid
TC-02	Validate journey	Existing journey ID	Valid
TC-03	Validate seat	Existing seat ID	Valid
TC-04	Retrieve available seats	Valid journey ID	Available seats returned
TC-05	Book available seat	Available seat	Booking confirmed
TC-06	Book unavailable seat	Booked seat	Booking rejected
TC-07	Invalid passenger	Unknown passenger ID	Request rejected
TC-08	Invalid journey	Unknown journey ID	Request rejected
TC-09	Invalid seat	Unknown seat ID	Request rejected
TC-10	Update seat status	Valid seat ID	Status updated

9.3 Integration Test Cases

Test ID	Test Description	Expected Result
IT-01	Frontend sends booking request	Backend receives request
IT-02	Backend checks seat availability	Correct seat status returned
IT-03	Booking service saves booking	Booking stored in database
IT-04	Booking updates seat status	Seat marked BOOKED
IT-05	Booking confirmation returned	Correct response displayed
IT-06	Database transaction fails	Changes rolled back
IT-07	Duplicate confirmed booking attempted	Database rejects duplicate

9.4 Concurrency Test Cases

Test ID	Test Description	Expected Result
CT-01	Two passengers request the same last seat	Only one booking succeeds
CT-02	Ten passengers request the same seat	Only one booking succeeds

CT-03	Fifty passengers request the same seat	Only one booking succeeds
CT-04	Multiple passengers request different seats	Valid bookings succeed
CT-05	Database failure during booking	Transaction rolled back
CT-06	Duplicate request with same idempotency key	No duplicate booking created

9.5 Performance Test Cases

Test ID	Test Description	Target
PT-01	100 simultaneous booking attempts	Measure response time and errors
PT-02	500 simultaneous booking attempts	Measure throughput
PT-03	1,000 simultaneous booking attempts	Measure response time and contention
PT-04	10,000 booking attempts in one minute	Evaluate increasing load
PT-05	50,000 booking attempts in one minute	Evaluate assignment scenario
PT-06	Peak-load response time	95% under 2 seconds

The tests must record actual results. A test target is not a guarantee that the system will pass.

9.6 Security Test Cases

Test ID	Test Description	Expected Result
ST-01	Invalid passenger input	Request rejected
ST-02	SQL injection input	Query remains safe
ST-03	Unauthorised admin access	Access denied
ST-04	Missing required fields	Validation error
ST-05	Duplicate booking request	No duplicate confirmed booking

9.7 Test Result Measurements

The following measurements will be collected:

1. Total booking requests.
2. Successful bookings.
3. Rejected bookings.
4. Duplicate confirmed bookings.
5. Average response time.
6. 95th percentile response time.
7. Maximum response time.
8. Requests per second.
9. Error rate.
10. Database transaction failures.

Expected outcome: A test plan covering functional, integration, concurrency, performance and security requirements.

STEP 10: DESIGN REVIEW AND VALIDATION

10.1 Introduction

The final step verifies that all design components are consistent with the requirements established in Phase 1 and the architecture defined in Phase 2.

10.2 Requirement Traceability Matrix

Requirement ID	Requirement	Design Element	Test ID
FR-01	Display available seats	Seat Service	TC-04
FR-02	Select available seat	Seat Selection UI	IT-01
FR-03	Accept multiple booking requests	Booking Service	CT-01
FR-04	Verify seat availability	Seat Service	TC-06
FR-05	Apply concurrency control	Lock Manager	CT-01
FR-06	Prevent duplicate booking	Unique DB Index	CT-01
FR-07	Update seat status	Database Manager	TC-10
FR-08	Confirm booking	Booking Service	IT-05

FR-09	Reject unavailable seats	Booking Service	TC-06
FR-10	Maintain booking records	Database Layer	IT-03
NFR-01	95% under 2 seconds	Performance Monitoring	PT-06
NFR-02	50,000 attempts/minute	Load Testing	PT-05
NFR-03	Zero duplicate bookings	Concurrency Control	CT-01
NFR-04	Data consistency	Database Transactions	IT-06
NFR-05	Protect booking information	Security Design	ST-01 to ST-05
NFR-06	Maintainable code	Modular Architecture	Design Review
NFR-07	Availability during peak demand	Deployment and Monitoring Design	PT-05
NFR-08	Simple interface	UI Design	UI Acceptance Testing

10.3 Design Review Checklist

No.	Review Activity	Status

1	Project folder structure defined	Completed
2	Database tables defined	Completed
3	Primary and foreign keys defined	Completed
4	Duplicate booking constraint defined	Completed
5	Classes and responsibilities defined	Completed
6	Function specifications prepared	Completed
7	Booking algorithm defined	Completed
8	Concurrency algorithm defined	Completed
9	API endpoints defined	Completed
10	UI screens designed	Completed
11	Functional test cases prepared	Completed
12	Concurrency test cases prepared	Completed
13	Performance test cases prepared	Completed

14	Security test cases prepared	Completed
15	Requirement traceability completed	Completed

10.4 Design Limitations

The following limitations must be considered before implementation:

1. SQLite is intended for the prototype and may become a bottleneck under heavy concurrent writes.
2. Python threading.Lock protects only threads in one application process.
3. A distributed deployment requires database-level concurrency protection across all workers.
4. The 50,000-attempts-per-minute target must be evaluated using load testing.
5. A 95th-percentile response time below two seconds must be measured rather than assumed.
6. Authentication and administrative access controls must be implemented before exposing the system beyond a controlled demonstration.

10.5 Phase 3 Deliverables

The following deliverables have been prepared:

1. Project folder structure.
2. Database schema and SQL design.
3. Class-level specifications.
4. Function-level specifications.
5. Seat allocation algorithm.
6. Concurrency control algorithm.
7. API-level specifications.
8. User interface wireframes.
9. Unit test cases.
10. Integration test cases.
11. Concurrency test cases.
12. Performance test cases.
13. Security test cases.
14. Requirement traceability matrix.
15. Design review checklist.

10.6 Conclusion

Phase 3 has converted the system architecture into detailed implementation specifications.

The project now has a proposed folder structure, database schema, class definitions, function specifications, booking algorithm, concurrency control strategy, API design, interface layouts and test cases.

The most important design decision is to combine application-level locking, database transactions and a unique constraint to prevent duplicate confirmed bookings.

These specifications will guide the development of the actual Railway Reservation System in Phase 4 – Implementation.

END OF PHASE 3